

atm_cell.vr

File location: \$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/atm/atm_cell.vr

```
class atm_cell extends rvm_data {
    static rvm_log log = new("atm_cell", "class");
    rand bit [ 3:0] gfc;
    rand bit [ 7:0] vpi;
    rand bit [15:0] vci;
    rand bit [ 2:0] pt;
    rand bit      clp;
    rand bit [ 7:0] hec;          // 00 = valid
    rand bit [ 7:0] payload[48];

    constraint hec_valid {
        hec == 8'h00;    // mask field for HEC, 1s indicate bad position
    }
    //
    // RVM Methods for this class
    //
    task new();
    virtual function string psdisplay(string prefix = "");

    virtual function bit is_valid(bit      silent = 1,
                                   integer kind   = -1);

    virtual function rvm_data copy(rvm_data to = null);

    virtual function bit compare(rvm_data to,
                                   var string diff,
                                   integer kind = -1);

    virtual function integer byte_size(integer kind = -1);

    virtual function integer byte_pack(var bit [7:0] bytes[*],
                                         integer offset = 0,
                                         integer kind    = -1);
    virtual function integer byte_unpack(bit [7:0] bytes[*],
                                         integer offset = 0,
                                         integer kind    = -1);

    virtual task save(integer file);
```

```

    virtual function bit load(integer file);

    //
    // Additonal Methods
    //
    virtual function bit[7:0] compute_hec();
}
rvm_channel(atm_cell)

task atm_cell::new()
{
    super.new(this.log);
    hec = 0;    // by default assume HEC is to be valid
}

function bit atm_cell::is_valid(bit        silent = 1,
                                integer kind  = -1)
{
    is_valid = 1;
    if (this.hec != 8'h00) {
        is_valid = 0;
        if (!silent) rvm_warning(this.log, "HEC value is invalid");

        return;
    }
}

function string atm_cell::psdisplay(string prefix = "")
{
    integer i;
    sprintf(psdisplay, "%sATM cell #%0d.%0d.%0d\n", prefix,
            this.stream_id, this.scenario_id, this.object_id);
    sprintf(psdisplay, "%s%s   gfc:0x%h vpi:0x%h vci:0x%h pt:0x%h clp:0b%b hec:%0s\n", p
            prefix,    gfc,        vpi,        vci,        pt,        clp,
            (hec) ? "BAD" : "good");
    sprintf(psdisplay, "%s%s   data:", psdisplay, prefix);
    for (i = 0; i < 4; i++) sprintf(psdisplay, "%s 0x%h", psdisplay, payload[i]);
    sprintf(psdisplay, "%s ..", psdisplay);
    for (i = 44; i < 48; i++) sprintf(psdisplay, "%s 0x%h", psdisplay, payload[i]);
    sprintf(psdisplay, "%s\n", psdisplay);
}

function rvm_data atm_cell::copy(rvm_data to = null)
{
    atm_cell cpy;

```

```

// Copying to a new instance?
if (to == null) {
    cpy = new;
} else
// Copying to an existing instance. Correct type?
if (!cast_assign(cpy, to, CHECK)) { // upcast to -cpy
    rvm_fatal(this.log, "Attempting to copy to a non atm_cell instance");
    copy = null;
    return;
}
super.copy_data(cpy);

cpy.gfc = this.gfc;
cpy.vpi = this.vpi;
cpy.vci = this.vci;
cpy.pt  = this.pt;
cpy.clp = this.clp;
cpy.hec = this.hec;
foreach(payload, i)
    cpy.payload[i] = this.payload[i];
copy = cpy;
}

function bit atm_cell::compare(rvm_data to,
                               var string diff,
                               integer kind = -1)
{
    atm_cell cmp;
    compare = 1; // Assume success by default.
    diff    = "No differences found";

    if (!cast_assign(cmp, to, CHECK)) {
        rvm_fatal(this.log, "Attempting to compare to a non atm_cell instance");
        compare = 0;
        diff = "Cannot compare non atm_cell to atm_cell";
        return;
    }

    // data types are the same, do comparison:
    if (this.gfc != cmp.gfc) {
        diff = psprintf("Different GFC values: %b != %b", this.gfc, cmp.gfc);
        compare = 0;
        return;
    }

```



```

integer kind = -1)
{
    bit [7:0] hec;
    // Make sure there is enough room in the array
    if (bytes.size() < this.byte_size() + offset) {
        bytes = new [this.byte_size() + offset] (bytes);
    }
    hec = compute_hec() ^ this.hec;
    // vera_pack()'s offset is in *bits*
    offset *= 8;
    void = vera_pack(bytes, offset, gfc, this.vpi, this.vci, this.pt, this.clp, this.hec,
        this.payload);

    byte_pack = 53;
}

function integer atm_cell::byte_unpack(bit [7:0] bytes[*],
integer offset = 0,
integer kind = -1)
{
    bit [7:0] hec;
    // Make sure there will be enough data to unpack
    if (bytes.size() < this.byte_size() + offset){
        bytes = new [this.byte_size() + offset] (bytes);
    }
    // vera_unpack()'s offset is in *bits*
    offset *= 8;
    void = vera_unpack(bytes, offset, this.gfc, this.vpi, this.vci, this.pt, this.clp, this.h
        this.payload);

    this.hec = compute_hec() ^ hec;
    byte_unpack = 53;
}

task atm_cell::save(integer file)
{
    bit [7:0] bytes[*];
    integer i;

    void = this.byte_pack(bytes);
    foreach (bytes, i) {
        fprintf(file, "%x", bytes[i]);
    }
    fprintf(file, "\n");
}

```

```

}

function bit atm_cell::load(integer file)
{
    bit [7:0] bytes[*];
    string line;
    integer i;
    line = freadstr(file);
    if (line == null) {
        load = 0;
        return;
    }

    bytes = new [53];
    foreach (bytes, i) {
        string image = line.substr(i * 2, i * 2 + 1);
        bytes[i] = image.atohex();
    }
    void = this.byte_unpack(bytes);
    load = 1;
}

function bit[7:0] atm_cell::compute_hec()
{
    bit [31:0] crc;
    //
    // This ATM HEC computation is believed to be correct
    // but no ganrantee of correctness or fit for any use
    // is implied or offered
    //

    crc = {gfc,vpi,vci,pt,clp} % 9'b1000000111;
    crc = crc ^ 8'b0101_0101;
    compute_hec = crc;
}

```